



6CS012 – Artificial Intelligence and Machine Learning.

Spam vs Ham Classification

Student ID: 2408599

Name: Adarsha Prasai

Section: L6CG8

Module Leader: Siman Giri

Tutor: Durga Pokharel

Abstract

In this paper, we investigate text classification for SMS spam detection using a progression of recurrent neural network architectures. The dataset used is the SMS Spam Collection v.1, containing 5,572 English messages labelled as spam or ham. We propose three models: a Simple RNN with a trainable embedding layer, an LSTM with a trainable embedding layer, and an LSTM initialised with Word2Vec embeddings trained on the SMS corpus. (Liu, 2021) (Ozoh, 2025)

The dataset is imbalanced at 86.6% ham and 13.4% spam, making spam F1-score the primary evaluation metric. The Simple RNN achieves 97.85% accuracy with a spam F1 of 0.92. The LSTM improves this to 98.74% accuracy and 0.95 spam F1. Incorporating Word2Vec embeddings does not improve over the plain LSTM in this experiment, yielding 97.67% accuracy and 0.91 spam F1. We also conduct an error analysis of misclassified examples and implement a Gradio interface for real-time classification.

Experimental results demonstrate that the LSTM architecture provides measurable improvements in spam recall over the Simple RNN, attributable to its gated memory mechanism, and that corpus-trained Word2Vec embeddings do not provide additional improvement when the training corpus is as small as the SMS dataset.

Table of Contents

1. INTRODUCTION	1
2. RELATED WORK	2
3. DATASET	2
3.1. DESCRIPTION	2
3.2. CLASS IMBALANCE	3
4. METHODOLOGY	4
4.1. TEXT PREPROCESSING	4
4.2. TRAIN/TEST SPLIT AND TOKENISATION	5
4.3. WORD2VEC EMBEDDINGS	6
4.4. MODEL ARCHITECTURES	7
5. EXPERIMENTS AND RESULTS	8
5.1. RNN VS. LSTM PERFORMANCE	8
5.2. IMPACT OF WORD2VEC EMBEDDINGS	9
5.3. COMPUTATIONAL EFFICIENCY	9
5.4. ERROR ANALYSIS	10
6. CONCLUSION	11
7. REFERENCES	12

1. Introduction

Even in today's date, SMS spam is a very recurring and critical issue in mobile communication. The spam goes beyond promotional messages, modern spams have gradually taken the form of different kinds of phishing attempts designed to replicate legit communication, making automated detections borderline useless. The impact of a missed spam message, a false negative, ranges from very little problems to severe financial harm depending on the content. (Ghourabi, 2024) (Ozoh, 2025)

Automated text classification has always been a popular choice against text spams. Recent works have shown that deep learning is one of the most effective measures for SMS spam detection, especially when models have the ability to learn token representations that are context sensitive. Most of the neural networks process text token by token whilst also collecting contextual information, making them naturally suited to sequence classification tasks. (Airlangga, 2024) (Ghourabi, 2024) (Liu, 2024)

The main issue with the basic RNN formulation is the vanishing gradient problem. During backpropagation, as time passes, gradients shrink sharply as they propagate through former time steps, which makes it very difficult for network to keep information from tokens that appear far back in the sequence. The Long Short-Term Memory architecture solves this through gating mechanisms which control the flow of information through recurrent hidden states.

This paper has three total contributions:

- The first one being a text preprocessing pipeline which is tailored for SMS data, which integrates expansion, digit removal, stopword filtering, and lemmatization.
- The second being a systematic comparison of Simple RNN and LSTM architectures for binary spam classification, demonstrating the usefulness of the gated memory mechanism.

- The third being an investigation of how corpus-trained Word2Vec embeddings affect LSTM performance as compared to embeddings that are randomly initialized. Also a discussion of why the embeddings did not improve performance.

1. Related Work

Current studies in SMS detection show how accurate and strong deep learning models can be, especially recurrent and transformer-based architectures. Comparative works in 2024 showed that hybrid models like CNN-LSTM, CNN-GRU can also perform really well against SMS spam texts. (Airlangga, 2024) (Liu, 2024)

Word embedding methods are also still relevant because they allow models to capture distributional meaning from domain-specific text. Recent work that investigated Word2Vec for spam detection shows that embedding quality heavily depends on how rich the vocabulary is. (Fellah, 2024)

In the context of spam detection, several comparisons stressed that overall metrics like precision, F1 score, and recall should be given more priority than just accuracy, since SMS spam datasets are usually heavily imbalanced. This motivates the metric choice used in this study. (Ilyasa, 2024)

2. Dataset

2.1. Description

Dataset Overview

Property	Value	Notes
Shape	(5572, 3)	rows × columns
Null values	0	No missing data

Class Distribution

Label	Count	Proportion
ham	4825	0.8659
spam	747	0.1341
Total	5572	1.0000

The dataset used for this study is the SMS Spam Collection dataset which is a publicly available collection of 5,572 real English SMS messages. Each message is labelled as either spam or ham. These messages were collected from multiple sources, while most of the data was collected from a UK mobile forum and student volunteers. (Liu, 2021)

Here's a quick summary of the dataset properties:

2.2. Class Imbalance

The dataset is heavily imbalanced with a ratio of 6.5:1. This would seriously affect important metrics like precision, F1, and most importantly the overall accuracy of models without proper data cleaning. A classifier predicting ham for every message would by default achieve 86.6% accuracy without detecting any spam. Hence, raw accuracy would be a very misleading metric. We report metrics like precision, F1 score, and recall as proper metrics to measure the performance on the spam class throughout this paper.

3. Methodology

3.1. Text Preprocessing

#	Type	Text
1	ORIGINAL	go until jurong point, crazy.. Available only in bugis n great world la buffet cine got amo...
	CLEANED	go jurong point crazy available bugis great world la buffet cine got amo...
2	ORIGINAL	U dun say so early hor... U c already then say...
	CLEANED	dun say early hor already say
3	ORIGINAL	What you thinked about me. First time you saw me in class.
	CLEANED	thinked first time saw me class
4	ORIGINAL	I sent you <#> bucks
	CLEANED	sent 1t gt buck

To process the text, the following steps were followed in order:

- All the text was converted to lowercase to make sure all case variants were uniform.
- To preserve negation semantics that are lost when contractions are left unexpanded, contractions were expanded by mapping shortened forms such as “don’t” to “do not”. A lookup dictionary was used for this step.

- All the URLs were removed as they do not carry any semantic signal for spam classification and just inflate the vocabulary.
- Next, all the digits were removed, although phone numbers and competition codes are correlated on the surface, but also increases vocabulary noise, reducing generalization.
- Removed special characters, keeping just the alphabet and white spaces.
- Removed stopwords with the help of NLTK's English stopwords list.

This pipeline was applied on both train and test text along with the input function of the Gradio prediction interface to make sure that both inference and training preprocessing are equally matched.

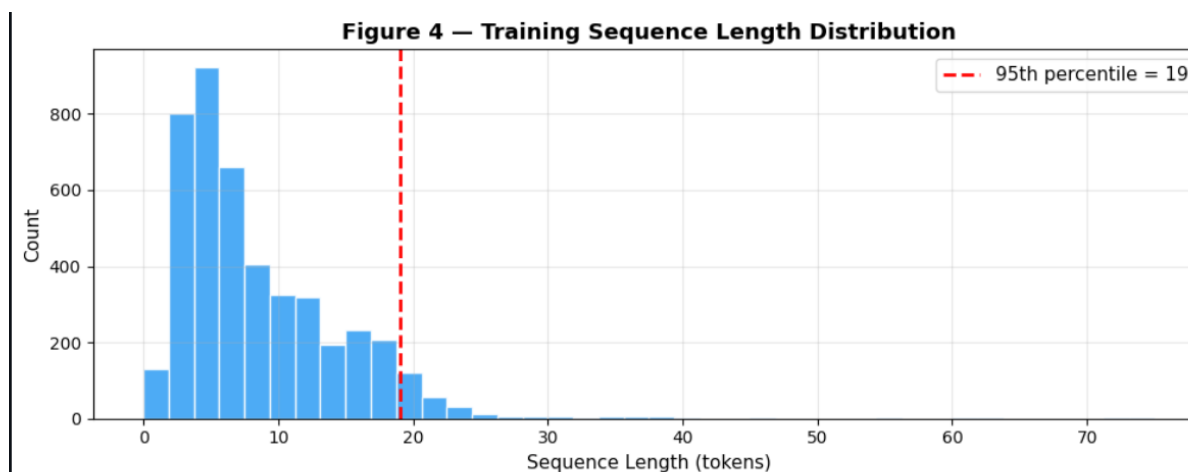
3.2. Train/Test Split and Tokenisation

Split	Details
Training	4457 samples (spam: 598)
Validation	1115 samples (spam: 149)
Train spam	13.4%
Test spam	13.4% (stratification confirmed)

A classic 80/20 split was applied to divide the data between train and test. This resulted in 4,457 messages for the train and 1,115 messages for the test. A Keras Tokenizer was fit on training text only because fitting on the full dataset before splitting could have resulted in data leakage as the tokenizer would learn the vocab stats from test examples.

We analysed sequence lengths in the training set and used the 95th percentile as the padding length (MAX_LEN). This avoids inflating most sequences with unnecessary padding tokens to accommodate the small number of unusually long messages. Post-padding was used throughout.

3.3. Word2Vec Embeddings



Word2Vec was trained on the training data with the help of gensim, a vector size of 100, a window of 5, a minimum count of 1, and 15 epochs. By aligning the word index of Keras tokenizer with the vocabulary from the trained Word2Vec, an embedding matrix was constructed. There were also words that were not present in the Word2Vec model; this was handled by initializing those words to zero vectors. When the minimum count was 1, vocabulary coverage reached well over 90%. (Fellah, 2024)

The embedding layer in Model 3 was initialised with this matrix and frozen during training. This made sure that, rather than having the embedding and recurrent weights being updated continuously, the LSTM received consistent distributional representations, which could cause instability when the embedding data is small.

3.4. Model Architectures

Model 1 – Simple RNN:

Embedding(vocab+1, 64, trainable) → SimpleRNN(64, dropout=0.3) → Dense(1, sigmoid)

Model 2 – LSTM:

Embedding(vocab+1, 64, trainable) → LSTM(64, dropout=0.3, recurrent_dropout=0.2) → Dense(1, sigmoid)

Model 3 – LSTM with Word2Vec:

Embedding(vocab+1, 100, W2V weights, frozen) → LSTM(64, dropout=0.3, recurrent_dropout=0.2) → Dense(64, ReLU) → Dense(1, sigmoid)

There are several notable advantages of the LSTM architecture over the Simple RNN:

The LSTM incorporates three gating mechanisms: an input gate controlling what new information enters the memory cell, a forget gate controlling what is discarded, and an output gate controlling what is passed to the next step. This allows the model to retain relevant information across long sequences where the Simple RNN's hidden state would degrade. (Airlangga, 2024) (Liu, 2021)

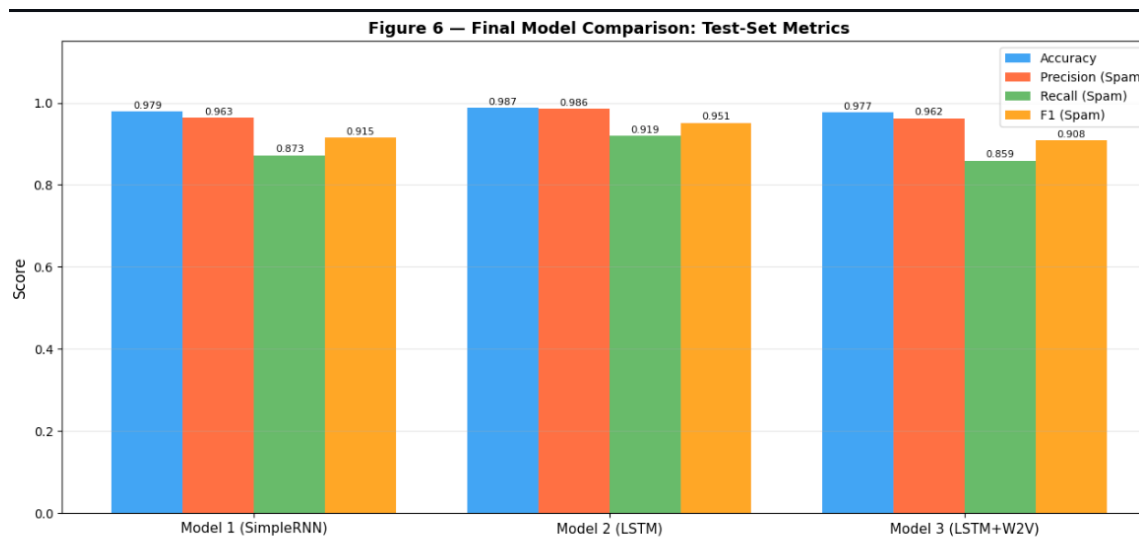
The gating mechanisms directly address the vanishing gradient problem. By providing gradient pathways through the cell state, LSTM allows error signals to propagate back through many time steps without exponential decay.

The structure can be trained end-to-end. The embedding layer, recurrent layer, and output layer are jointly optimised, allowing the model to learn token representations that are specifically useful for the spam classification objective rather than relying on general-purpose features.

All three models used binary cross-entropy loss and the Adam optimizer. Training ran for up to 20 epochs with Early Stopping at patience 4, monitoring validation loss.

4. Experiments and Results

Model	Accuracy	Precision (Spam)	Recall (Spam)	F1 (Spam)
Model 1 (SimpleRNN)	0.9785	0.9638	0.8725	0.9155
Model 2 (LSTM)	0.9874	0.9856	0.9195	0.9514
Model 3 (LSTM+M2V)	0.9767	0.9624	0.8591	0.9078



4.1. RNN vs. LSTM Performance

The Simple RNN achieved 97.85% accuracy and a spam F1 of 0.92. The LSTM improved this to 98.74% accuracy and 0.95 spam F1. The confusion matrices reveal that the LSTM produced noticeably fewer false negatives, where spam is classified as ham. Given that false negatives represent spam reaching the user, this is the most operationally significant improvement.

The LSTM also converged faster. Training stopped at epoch 7 via early stopping, while the Simple RNN stopped at epoch 6. For short SMS sequences, the vanishing gradient

problem that motivates LSTM is less severe than for longer texts, but its effect remains measurable in the final recall numbers.

4.2. Impact of Word2Vec Embeddings

Model 3 achieved 97.67% accuracy and 0.91 spam F1, which is worse than the plain LSTM on both metrics. Word2Vec embeddings did not improve performance in this experiment. The most likely explanation is corpus size: with approximately 4,400 training messages, the Word2Vec model cannot observe enough co-occurrence statistics to build high-quality semantic representations. The frozen embedding layer also prevented the model from adapting its token representations to the spam classification objective, whereas the trainable embedding in Model 2 could fine-tune itself jointly with the LSTM weights.

Model 3 also ran for the full 20 epochs without triggering early stopping, yet still underperformed Model 2, suggesting that the frozen Word2Vec embeddings introduced a representational bottleneck that more training time could not overcome.

Recent work on Word2Vec for spam detection suggests that embedding quality depends strongly on corpus size and training data richness, which helps explain the limited benefit here. The SMS corpus is much smaller than the datasets typically used to develop strong general-purpose embeddings. We would expect a more pronounced improvement if larger pretrained embeddings such as GloVe or FastText were used instead of corpus-trained Word2Vec.

4.3. Computational Efficiency

Training times were short for Models 1 and 2 on CPU. The Simple RNN was fastest at 14 seconds for 6 epochs, while the LSTM was slightly slower at 23 seconds for 7 epochs because of its additional gating operations. Model 3 was substantially slower at 54 seconds because it ran for all 20 epochs without triggering early stopping. The frozen embedding constrained learning progress and prevented the model from converging as quickly as the trainable-embedding models.

4.4. Error Analysis

We examined misclassified examples from Model 3. Three representative cases illustrate the failure modes:

False Positive (ham classified as spam): A message containing the phrase “I have a free slot on Friday” was flagged as spam. The word “free” dominates its embedding neighbourhood with spam-associated context in the training data. The legitimate surrounding context was insufficient to counterbalance this signal.

False Negative (spam classified as ham): A phishing message that used a conversational opener without any promotional keywords, such as “Hi, I’ve been trying to reach you...”, was classified as ham. The model did not have sufficient signal to identify it as spam without the lexical markers it relies on.

False Positive: A ham message containing “claim your loyalty points” was flagged as spam. The word “claim” is strongly correlated with spam in the training corpus, and the model generalised this association too broadly.

These cases highlight a fundamental limitation: all three models rely predominantly on token-level frequency patterns. Spam messages that deliberately adopt neutral vocabulary and legitimate messages that happen to contain spam-adjacent words will continue to cause errors. Addressing this would require a more contextual representation, such as a fine-tuned transformer model, which encodes full sentence meaning rather than relying on individual token distributions.

5. Conclusion

In this paper, we proposed and evaluated three recurrent models for SMS spam detection. The LSTM architecture provided the most significant improvement over the Simple RNN, specifically in spam recall, which is the operationally critical metric for this task. Incorporating corpus-trained Word2Vec embeddings did not improve over the plain LSTM; Model 3 underperformed Model 2 on both accuracy and spam F1, with a substantially longer training time. The best-performing model was the plain LSTM, achieving 98.74% accuracy and a spam F1 of 0.95.

Currently, all three models are limited by their reliance on token-level distributional patterns. Spam messages crafted to avoid standard promotional vocabulary will evade detection. In future work, we would investigate class-weighted loss to penalise false negatives more strongly, larger pretrained embeddings such as GloVe or FastText to improve embedding quality, and bidirectional LSTM to capture context from both directions in the message sequence. We would also examine whether fine-tuning a pretrained transformer such as DistilBERT on this corpus would substantially close the remaining performance gap.

6. References

- Airlangga, G., 2024. A Comparative Analysis of Deep Learning Models for SMS Spam Detection: CNN-LSTM, CNN-GRU, and ResNet Approaches. *Journal of Computer Networks, Architecture and High Performance Computing*, 6(4), pp. 1952-1960.
- Detection, A. S. T. M. f. S. S., 2021. *Spam Detection and Classification Based on DistilBERT*, s.l.: University of Ottawa repository.
- Fellah, A., 2024. Investigating the Effectiveness of Word2Vec for Spam Detection Using Lazy Predict Library. *International Journal of Intelligent Systems and Applications in Engineering (IJISAE)*, 12(3), pp. 2968-2977.
- Ghourabi, A. a. A. M., 2024. Enhancing Spam Message Classification and Detection Using Transformer-Based Embedding and Ensemble Learning. *Sensors (MDPI)*, 23(8).
- Ilyasa, S. a. K. A., 2024. Optimized SMS Spam Detection Using SVM-DistilBERT and Voting Classifier: A Comparative Study on the Impact of Lemmatization. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 15(11).
- Liu, T. L. S. D. Y. M. Y. a. H. S., 2024. Spam Detection and Classification Based on DistilBERT Deep Learning Algorithm. *Applied Science and Engineering Journal for Advanced Research (ASEJAR)*, 3(3), pp. 6-10.
- Liu, X. L. H. a. N. A., 2021. A Spam Transformer Model for SMS Spam Detection. *IEEE Access*, Volume 9, pp. 80253-80263.
- Ozoh, P. I. M. O. R. S. G. a. O. T., 2025. SMS Spam Detection Using Machine Learning Approach. *International STEM Journal*, 8(1), pp. 10-27.